From Dirty Data to Predictive Models: Heart Disease Prediction

Heart disease is one of the most common causes of death in the world. Early identification of individuals at risk is really important. In this assignment, I built predictive models, which determine whether a patient has heart disease based on clinical measurements and diagnostic features. The dataset used in this assignment is the UCI Heart Disease dataset. It contains medical information from 303 patients, including personal information like age and sex, it also includes some other variables that might be the cause of heart disease. Although the original dataset encodes disease severity (heart disease) from 0 to 4, I transformed the task into a binary classification problem. Using the function df['num'].value_counts(), I can see that 164 people don't have heart disease. Whereas 139 people have, so after the transformation, I get:

- 0 indicates no heart disease
- 1 indicates presence of heart disease

These binary numbers simplify interpretation to the question of: Does the patient have heart disease or not? The goal of this assignment is not only to train predictive models but also try to walk through the entire machine learning pipeline: cleaning data, engineering features, training generative and discriminative models, evaluating performance, and interpreting results.

Data Cleaning

Before building any model, the dataset must be checked first. According to the dataset documentation, the variables ca (number of major vessels) and thal (thalassemia type) contain missing values. Before model training, the dataset was carefully inspected for missing values. Using df.isnull().sum(), I observed that two variables contained missing entries:

- ca had 4 missing values
- thal had 2 missing values

In total, 6 observations contained incomplete information. Since the proportion of missing data was very small, I chose to remove those rows using dropna(). This approach avoids introducing artificial assumptions through imputation and keeps the preprocessing transparent. After removing incomplete rows, 297 observations remained for modeling.

Feature Engineering

The dataset was divided into training and testing sets using an 80/20 split. Stratified sampling was applied to preserve the class distribution in both sets. This makes sure that the proportion of patients with and without heart disease remains consistent. Importantly, the split was performed before any encoding or scaling steps to avoid data leakage. Several variables in the dataset are categorical, even though they are stored numerically. For example, chest pain type (cp), slope of the ST segment (slope), and thalassemia (thal) represent categories rather than quantities. Treating them as continuous values would incorrectly imply numerical ordering. To address this, I applied one-hot encoding, converting each categorical variable into binary indicator columns. One category per variable was intentionally dropped to avoid redundancy and perfect linear dependence among features. After encoding, the total number of features increased from 13 to 20. No information was removed; it was simply reorganized into a format suitable for linear modeling. (Note: one column will be dropped as a reference group for categorical variables like chest pain type, this step will reduce the computing time for larger dataset, also, without a reference, the model tries to compare categories in a circular way. We know that if cp_2, cp_3,

and cp_4 are all false, then that patient will fall into the reference group). Finally, continuous variables such as age, cholesterol level, resting blood pressure, maximum heart rate, and ST depression were standardized. Standardization centers each variable around zero and scales it to unit variance.

Model Training

Two fundamentally different modeling approaches were implemented: a generative model (Gaussian Naive Bayes) and a discriminative model (Linear Regression). In addition, regularized versions of linear regression were explored to examine whether controlling model complexity would improve generalization. Gaussian Naive Bayes models each feature as following a normal distribution within each class. While it assumes independence between different features. It often performs surprisingly well in practice, particularly with smaller datasets (like this one). Two versions of the model were trained. The first used default smoothing parameters. The second increased the smoothing level to examine how added variance affects classification performance. The default model achieved an accuracy of approximately 83%. When smoothing was increased, accuracy dropped to 75%, which surprised me, because ideally smoothing is supposed to remove noise or reduce overfitting . However, in this case, this might be because the dataset is so small, given the fact that there are only 297 observations after cleaning. Smoothing inflates the variance of feature distributions, which can be helpful when data is sparse. The dataset here was already clean and standardized, so the default configuration is more appropriate.

```
NB default accuracy: 0.8333333333333334
NB smoother accuracy: 0.75
              precision    recall  f1-score   support

           0       0.81      0.91      0.85        32
           1       0.88      0.75      0.81        28

    accuracy                           0.83        60
   macro avg       0.84      0.83      0.83        60
weighted avg       0.84      0.83      0.83        60
```

Although linear regression is designed for predicting continuous values, it can be adapted for binary classification by applying a probability threshold. In this implementation, predicted outputs greater than 0.5 were classified as disease cases. The linear regression model achieved an accuracy of approximately 82%, slightly below Naive Bayes.

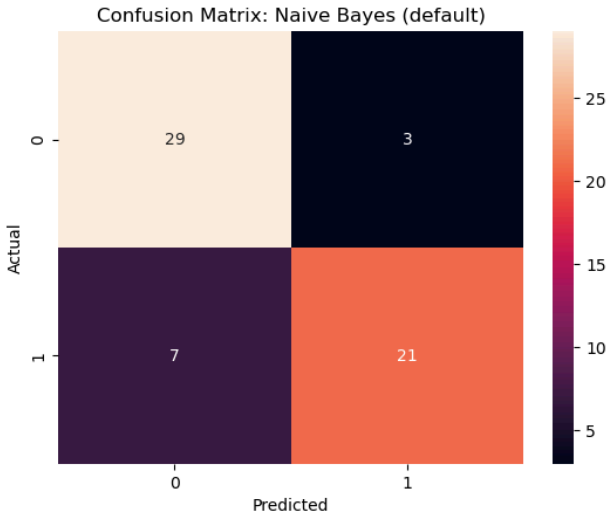
```
Linear Regression accuracy: 0.8166666666666667
              precision    recall  f1-score   support

           0       0.78      0.91      0.84        32
           1       0.87      0.71      0.78        28

    accuracy                           0.82        60
   macro avg       0.83      0.81      0.81        60
weighted avg       0.82      0.82      0.81        60
```

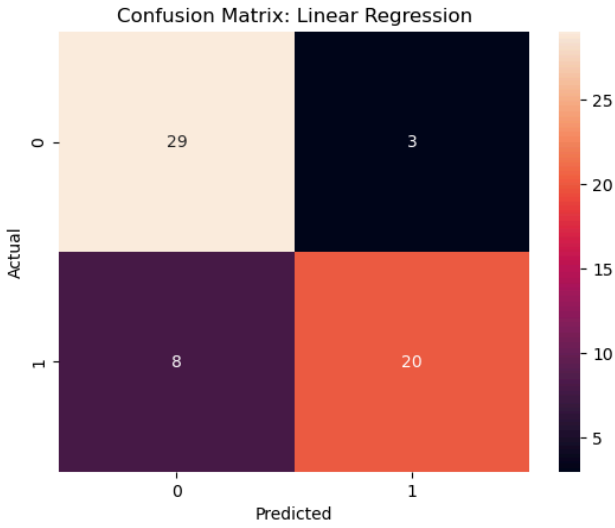
To explore the impact of regularization, Ridge (L2) and Lasso (L1) regression models were also trained. I would expect the Ridge regression model would perform better because it is good when small datasets are expected. Ridge regression yielded nearly identical performance to standard linear regression, while Lasso produced slightly lower accuracy (around 80%). This suggests that the model was not suffering from significant overfitting. The feature set was moderate in size, and multicollinearity did not appear severe enough to meaningfully degrade performance. Regularization therefore did not dramatically alter predictive outcomes.

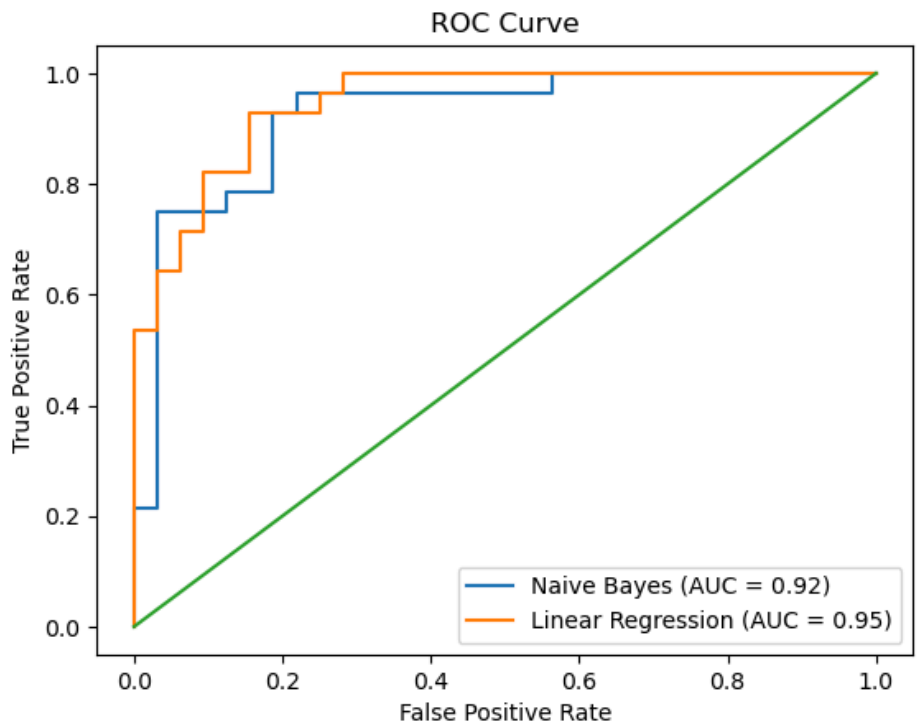
Model Evaluation

Model performance was evaluated using multiple metrics. Accuracy served as a primary metric. Naive Bayes with default smoothing performed best at 83%. Linear regression and Ridge followed closely at approximately 82%, while Lasso left slightly behind. While these differences are not extreme, the advantage of Naive Bayes suggests that its structure may align well with the characteristics of this dataset (small and clean).

Confusion matrices were generated to examine prediction patterns more closely. Across models, performance was slightly stronger for identifying patients without heart disease than for identifying those with disease. Recall for the disease class was lower than recall for the healthy class. Precision and recall scores confirmed that Naive Bayes maintained relatively balanced performance across classes, though disease recall remained somewhat lower. The F1-score, which balances precision and recall, supported the overall ranking observed in accuracy comparisons.


Confusion Matrix: Naive Bayes (default)
29
3
7
21
Actual
0
1
Predicted
0
1
25
20
15
10
5


Confusion Matrix: Linear Regression
29
3
8
20
Actual
0
1
Predicted
0
1
25
20
15
10
5



Increasing the smoothing parameter in Naive Bayes reduced performance. Larger smoothing values inflate estimated variance within each class, causing distributions to overlap more heavily. Because the dataset was already standardized, smoothing diminished the model's ability to distinguish between classes. This illustrates how model assumptions interact with data quality; adjustments that help in messy datasets may be not helpful at all in cleaner ones.

Conclusion

Overall, Naive Bayes slightly outperformed the linear models. Although its independence assumption is restrictive, the small size of the dataset and structured features appear to fit its framework. In smaller datasets, simpler models with lower variance can sometimes generalize more reliably.

Linear regression performed consistently but it is not as good as Naive Bayes. The improvement from regularization suggests that overfitting was not a dominant issue. Feature scaling and encoding likely contributed to stable model behavior.

There are several limitations though. The dataset is relatively small and may not fully capture the complexity of heart disease diagnosis. Converting the target into a binary outcome removes information about severity levels. Moreover, linear regression is not designed for classification, and more specialized models such as logistic regression or tree-based approaches could potentially improve performance.

AI Tool Usage Disclosure

I used ChatGPT and Gemini to help me include all of the Python libraries I needed because I'm not familiar with all of them. Also, I asked them to help me find the correct syntax in pandas. All data cleaning steps, model implementations, debugging, and performance analysis were conducted by myself. I included some comments on the side of the codes to reflect my own understanding of the workflow.

Code Link:

https://github.com/XinleiCheng/Machine_Learning_Portfolio/blob/main/Assignment1.ipynb